



CSI247: Data Structures

Lecture 06 – Recursion

Department of Computer Science

B. Gopolang (247-275)

Previous lesson

- Arrays
 - Advantages and disadvantages
 - Declaration
 - Memory diagram
 - Arrays of objects
 - Processing arrays

Today ...

Learning Objectives

- By the end of this lecture, you must be able to:
 - Differentiate between recursion and a recursive method
 - Understand 2 cases required when implementing a recursive method
 - Implement a recursive method
 - Trace a recursive method using a trace table
 - Convert a recursive method to a non-recursive version, and vice-versa

Implementing recursion

- Rem:
 - A recursive method calls itself to solve a problem
- A recursive method MUST implement 2 cases

a) Base case:

- Its purpose: to stop recursion
 - A simple case
 - Should be answered without using recursion

b) Recursion case: method must call itself (making a recursive call) to help compute the final answer

Example: factorial

- Often denoted as $n!$
- A classical example of recursion
- E.g.: $n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$
 - Defn: Product of all numbers between 1 & n , inclusive.
- Mathematically:

$$f(n) = \begin{cases} 1 & \text{if } n = 1 \text{ (Base case)} \\ n \cdot f(n-1) & \text{otherwise (Recursive case)} \end{cases}$$

$$= n * (n-1) * (n-2) * (n-3) * \dots * 1$$

$$\text{E.g. } 5! = 5 * 4 * 3 * 2 * 1 = 120$$

Implementing $n!$

a) Using iteration

● Rem $n! = n * (n-1) * (n-2) * (n-3) * \dots * 1$

```
public static long factL(int in) {  
    long prod = 1;  
    for (int i=1; i <= n; i++) {  
        prod = prod * i;  
    }  
    return prod;  
}
```

b) Using recursion

```
public static long factR(int n) {  
    long prod = 1;  
    if (n==1)  
        prod = 1;  
    else  
        prod = n * factR(n-1);  
    return prod;  
}
```

```
public static long factR(int n) {  
    if (n==1)  
        return 1;  
    else  
        return = n * factR(n-1);  
}
```

● NOTE:

- method factR is defined using another method factR

Important points...

- Base cases
 - Rem: purpose - to end/terminate the method
 - MUST NOT use recursion
 - Test all possible base cases (need at least 1)
- Recursive calls
 - Purpose: to advance closer to the base case
 - Taken care of by each iteration
 - Every possible chain of recursive calls must progress towards the base case
 - Where a method has several recursive options:
 - Only 1 recursive option must be executed

Executing a recursive method

Finally, cascade the returned values

- E.g.: $\text{factR}(5) \rightarrow 5$ is n when we get inside the method

```
if (n == 1) ← -----  
    return 1;  
else  
    return 5 * factR(4);  
    return 4 * factR(3);  
    return 3 * factR(2);  
    return 2 * factR(1);  
    return 1 (Base case reached!)
```

Cascading returned values

$$\begin{array}{c} 120 \\ \hline 5 * 24 \\ \hline 4 * 6 \\ \hline 3 * 2 \\ \hline 2 * 1 \end{array}$$

Tracing recursive methods

- Tracing factR(5) using a **trace table**

level	n	n==1?	Returned	Results
0	5	false	$5 * \text{factR}(4)$	$5 * 24 = 120$
1	4	false	$4 * \text{factR}(3)$	$4 * 6 = 24$
2	3	false	$3 * \text{factR}(2)$	$3 * 2 = 6$
3	2	false	$2 * \text{factR}(1)$	$2 * 1 = 2$
4	1	true	1	1

- Note:**

- Recursive method calls are stored in a stack

Pros and cons of recursion

● Advantages

- Shorter code
- Cleaner code

● Disadvantages

- Requires more memory space
- Intermediate calculations are stacked in memory
- Recursion is slightly slower
- Debugging may be difficult

Recursion vs Iteration

- Every recursion solution has a corresponding iterative solution
 - We can re-write from iterative to recursive, & vice versa
- Problem with recursion:
 - **Overheads** associated with multiple method calls.
 - However, for some problems recursive solutions are often **more simple and elegant** than iterative solutions.

Note:

- Need to determine when recursion is appropriate!

Infinite recursion

- Consider e.g. below:
- E.g.:

```
public long factR(int n) {  
    long prod;  
    prod = n * factR(n - 1);  
    return prod;  
}
```

- Will recursion stop?
 - No!
- Why?
 - Missing base case

Exercises

- a) Type *fact* & *factR* code segments provided in the handout. Fix errors, if any. Compile and run them.
- b) Is there any difference between output from the non-recursive and the recursive one?
- c) Write recursive method *power*, which takes int variables *x* and *y*, and returns x^y
- d) Convert method in c) to a non-recursive version
- e) Test methods in c) and d) above.
 - They **must** give the same output

- f) Write non-recursive method *fibonnacci*, that accepts a positive int value and display its fibonnacci series
- Fibonacci series: a sequence of numbers starting with 0 followed by 1, in which each number is the sum of its 2 preceding numbers
- g) Convert method in f) to its recursive version
- h) Trace the following method calls using trace tables:
- factR(7)
 - power(6, 3)
 - fibonacci(6) // *the recursive version*

Summary

- In this lesson, we were able to do these:
 - Differentiated between recursion and a recursive method
 - Identified the two cases required when implementing recursion
 - Implemented a recursive method
 - Traced a recursive method call using a trace table
 - Illustrated infinite recursion using an example

Next lesson

- Time complexity of algorithms

Q & A

Thank you

